

TD05 ATELIER : REPONSIVE DESIGN

Table des matières

1) Problématique.....	1
2) Exercices	1
3) Conclusion.....	10

1) Problématique

Nous cherchons à savoir comment nous pouvons créer une page HTML capable d'être utilisée sur ordinateur ET smartphone, sans pour autant faire une page HTML par support. Pour pouvoir faire cela nous allons réaliser plusieurs exercices qui permettront d'ajouter au code de notre HTML une capacité de responsive design qui permettra d'adapter le rendu de la page HTML par rapport à la taille de l'écran.

2) Exercices

Exercice 1 :

Pour effectuer cet exercice, nous allons utiliser la page HTML de Chuck Norris réalisé précédemment :

1) Nous allons changer la valeur de la largeur **width** pour la mettre à 100% puis nous allons rajouter l'attribut **auto** à **margin** en utilisant ce code :

```
body{
background-color:#838892;
width:100%;
line-height:150%;
text-indent:5px;
text-align:center;
margin:auto;
-}
```

2) Puis nous allons modifier le **<aside>** pour lui enlever la marge de gauche de 10% et lui changer sa largeur de 30% à 33% et le **<article>** pour lui changer sa largeur De 60% à 67% :

```

article {
  width:67%;
  background-color:#CCC;
}
aside {
  width:33%;
  background-color:#CCC;
}

```

Exercice 2 :

1) Ici nous allons ajouter une largeur maximum de 500px à **<article>** et de 250px à **<aside>** en utilisant la propriété CSS **max-width** :

Code :

```

article {
  width:67%;
  background-color:#CCC;
  max-width:500px;
}
aside {
  width:33%;
  background-color:#CCC;
  max-width:250px;
}

```

Résultat :

Chuck Norris						
<i>Biographie (From Wikipedia.)</i> <i>L'enfance</i>  Carlos Ray Norris, plus connu en tant que Chuck Norris, est un acteur américain né le 10 mars 1940 à Ryan (Oklahoma).	Auteurs	Karaté	Taekwondo	Judo	Chun Kuk Do	Tangsudo Ju-jitsu
	Chuck Norris	5	5	5	5	5
	Steven Seagal	3	5	3	2	5
	Bruce Lee	5	3	3	4	3
	Jean-Claude Van Damme	3	5	3	3	3
	Bolo Yeung	2	4	2	4	5

2) Puis nous allons faire la même chose mais avec une limite minimum (**min-width**) de 250px pour **<article>** et 150px pour **<aside>** :

```

article {
  width:67%;
  background-color:#CCC;
  max-width:500px;
  min-width:200px;
}
aside {
  width:33%;
  background-color:#CCC;
  max-width:250px;
  min-width:150px;
}

```

(Aucune différence actuel sur le résultat donc je ne l'affiche pas)

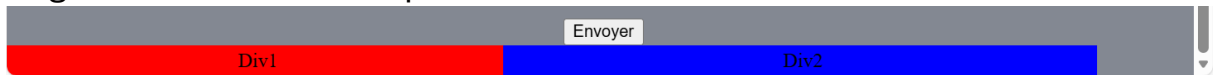
Exercice 3 :

Nous allons maintenant ce que fait le mélange des propriétés min-width et max-width avec les tailles en pourcentages :

1) Lorsqu'on met la page à 800px ou moins div1 et div2 prennent toute la largeur :



2) Par contre lorsque la page dépasse cette taille div1 va rester bloquer à sa largeur maximum de 400px :



Code utilisé :

```

<div style="display: flex;">
  <div style="width: 50%; max-width: 400px; background-color: red;">Div1</div>
  <div style="width: 50%; background-color: blue;">Div2</div>
</div>

```

Exercice 4 :

1) Je vais utiliser ce site pour m'aider :

https://www.w3schools.com/css/css3_flexbox_items.asp

2) Nous allons maintenant utiliser la propriété CSS **flex-grow** pour résoudre le problème d'avant :

Code :

```
<div style="display: flex;">
  <div style="width: 50%; max-width: 400px; background-color: red;">Div1</div>
  <div style="width: 50%; background-color: blue; flex-grow:1;">Div2</div>
</div>
```

Résultat :



flex-grow:1 prend le dessus sur **width:50%** car il permet d'occuper tout l'espace restant de l'espace vide sur la largeur de la page.

3) Maintenant que nous savons cela nous allons changer la largeur de **<article>** et **<aside>** pour les mettre respectivement à 300px et 200px. Puis nous allons rajouter les propriétés CSS **flex-grow** et **flex-shrink** qui seront de valeur **0** dans le code :

```
article {
  width:300px;
  background-color:#CCC;
  max-width:500px;
  min-width:200px;
  display:flex;
  flex-grow:0;
  flex-shrink:0;
}
aside {
  width:200px;
  background-color:#CCC;
  max-width:250px;
  min-width:150px;
  display:flex;
  flex-grow:0;
  flex-shrink:0;
}
```

Résultat :

Biographie (From Wikipè		Acteurs	Karaté	Taekwondo	Judo
L'enfance					
	Carlos Ray Norris, que Chuck Norr américain né le 10 (Oklah	Chuck Norris	5	5	5
Spécialiste en arts martiaux, il s'est essentiell					

En bougeant la page on s'aperçoit que rien ne change, sûrement car la valeur de **flex-grow** et **flex-shrink** sont à 0.

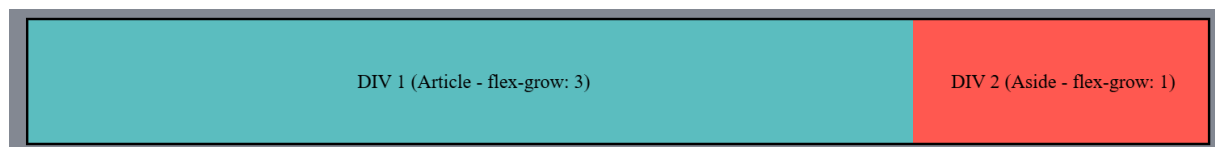
4) Pour que **<article>** soit 3 fois plus grand que **<aside>** il faut que la valeur de son **flex-grow** soit 3 fois plus grande ce qui signifie qu'il faut changer la valeur de **<aside>** à 1 et changer la valeur de **<article>** à 3 :

(J'ai eu des soucis avec le code donc j'ai eu à implémenter la propriété CSS **flex-basis**, qui sera à 0 ce qui signifie que le contenu de flex n'aura pas de largeur fixe, rien n'a marché dans le code principal donc j'ai fait une recreation des deux div d'avant) :

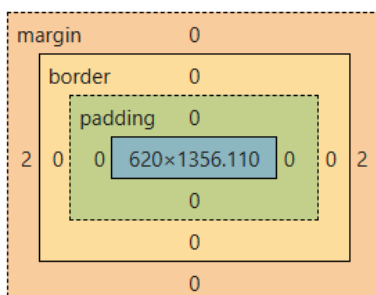
Code :

```
<div style="display: flex; width: 95%; margin: 20px auto; height: 100px; border: 2px solid black;">
  <div style="flex: 3 0 0; background-color: #5BBDBF; text-align: center; line-height: 100px;">
    DIV 1 (Article - flex-grow: 3)
  </div>
  <div style="flex: 1 0 0; background-color: #FF5850; text-align: center; line-height: 100px;">
    DIV 2 (Aside - flex-grow: 1)
  </div>
</div>
```

Résultat :



5) Pour faire cette question nous allons changer la taille du body dans l'inspecteur d'élément (étant donné qu'il n'y a pas de CSS dans la page HTML où j'ai créé ces deux div, ils ne remplissent pas toute la largeur mais le concept du ratio 3 : 1 se verra quand même) :



(Dans mon cas faire cela ne change rien mais ça sera utile dans un cas où il y a un CSS)

Code :

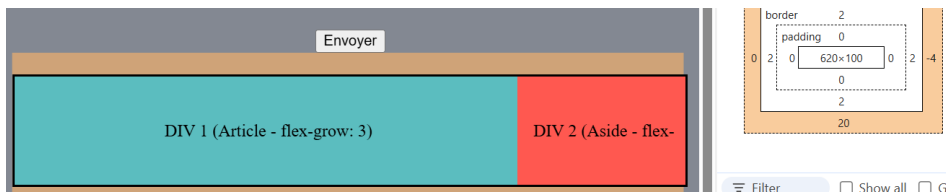
```
<div style="display: flex; width: 95%; margin: 20px auto; height: 100px; border: 2px solid black;">

  <div style="flex: 3 0 0; background-color: #5BBDBF; line-height: 100px;">
    DIV 1 (Article - flex-grow: 3)
  </div>

  <div style="flex: 1 0 0; background-color: #FF5850; line-height: 100px;">
    DIV 2 (Aside - flex-grow: 1)
  </div>

</div>
```

Nous allons donc simplement changer la largeur de l'ensemble des deux div :



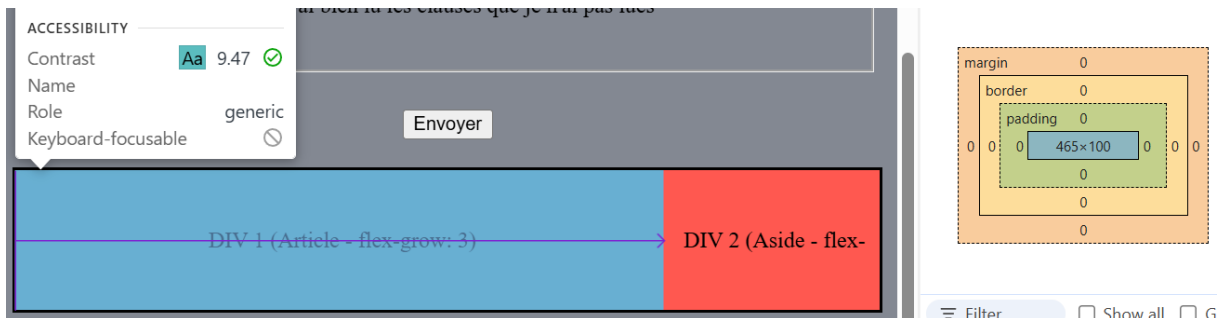
DIV 1 occupe $\frac{3}{4}$ et DIV 2 $\frac{1}{4}$. Donc DIV 1 doit faire $\frac{3}{4} * 620$ et DIV 2 $\frac{1}{4} * 620$:

DIV 1 doit donc faire : 465px

DIV 2 doit donc faire : 155px

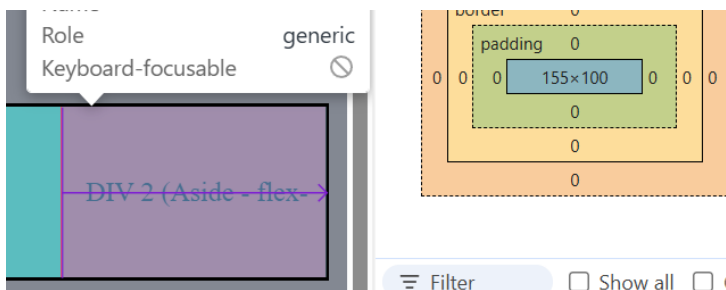
Résultats :

DIV 1 :



C'est bon

DIV 2 :



C'est bon aussi.

6) DIV 1 diminue des deux tiers (2/3) de la largeur à supprimer et DIV 2 diminue du reste (1/3), on doit utiliser la propriété CSS flex-shrink. Pour implémenter ce ratio de 2 : 1 sur la réduction :

Code :

```
<div style="display: flex; width: 95%; margin: 20px auto; height: 100px; border: 2px solid black;">

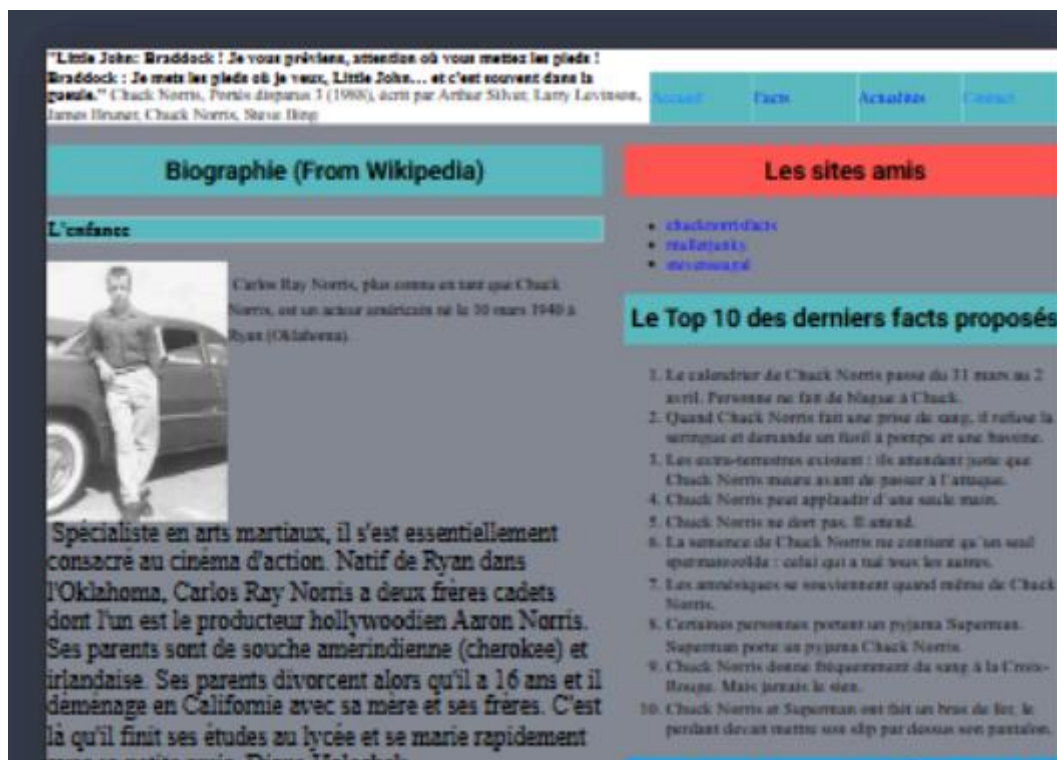
  <div style="flex: 3 2 0; background-color: #5BBDBF; line-height: 100px;">
    DIV 1 (Article - flex-grow: 3)
  </div>

  <div style="flex: 1 1 0; background-color: #FF5850; line-height: 100px;">
    DIV 2 (Aside - flex-grow: 1)
  </div>

</div>
```

Exercice 5 :

1) Exemple de résultat sur téléphone :



2)

360 × 740 100% ▼

Exercice 6 : En ajoutant cette ligne de code :

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Le navigateur ne va plus chercher à essayer de garder la résolution de base , on demande au navigateur d'utiliser la vraie largeur de l'écran.

Exercice 7 :

Le lien ne marche pas.

Exercice 8 :

1) Pour pouvoir faire cela nous devons utiliser un système de condition en utilisant **and**, on veut enlever le tableau donc on le fait disparaître avec l'attribut **none** :

```
@media screen and (max-width: 768px) {  
  table {  
    display: none;  
  }  
}
```

2) Ici de même mais nous allons utiliser l'attribut **flex-direction** pour **main** :

```
@media screen and (max-width: 480px) {  
  main {  
    flex-direction: column;  
  }  
}
```

Exercice 9 :

1) Nous allons créer un **<div>** « burger » :

```
</div>  
<div class="burger">  
    
</div>
```

2) Maintenant dans le CSS nous allons cacher le menu par défaut avec **display:none** puis on va positionner l'image en haut à droite de la fenêtre :

```
.burger img {  
  position: fixed;  
  top: 10px;  
  right: 10px;  
  z-index: 1000;  
  cursor: pointer;  
}
```


3) Ensuite on implémente ce code avec d'autres choses en plus :

```
<div class="burger">
  
  <div id="menu2">
    <div><a href="/index.html">Accueil</a></div>
    <div><a href="/facts.html">Facts</a></div>
    <div><a href="/news.html">Actualités</a></div>
    <div><a href="/contact.html">Contact</a></div>
  </div>
</div>
```

Code :

```
#menu2 {
  position: fixed;
  top: 0;
  right: 0;
  width: 80vw;
  height: 100vh;
  background-color: rgba(91, 189, 191, 0.95);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 20px;
  z-index: 10;
  margin-right: -80vw;
}
```

4) Maintenant nous allons faire en sorte de faire apparaître le menu lorsqu'on survole le menu :

Code :

```
.burger:hover #menu2 {
  display: flex;
}
```

Résultat :



3) Conclusion

J'ai appris durant ce TP à modifier mon code pour qu'il puisse s'adapter à différents types d'appareils, c'était plutôt simple mais c'est devenu très difficile à la fin du TP je trouve.